



ENGENHARIA DE SOFTWARE

PROCESSOS – MÉTODOS ÁGEIS

PROFA. REBECA SCHROEDER FREITAS

REBECA.SCHROEDER@UDESC.BR

MANIFESTO ÁGIL (2001)

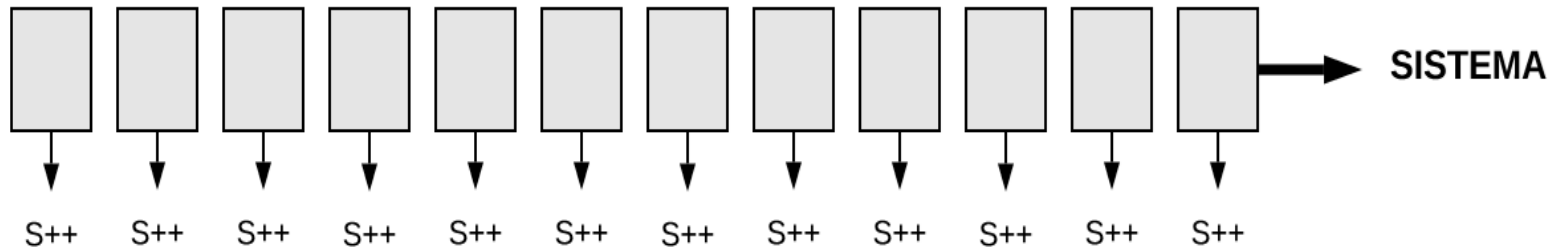


IDEIA CENTRAL: DESENVOLVIMENTO ITERATIVO

Waterfall



Ágil



OUTROS PONTOS IMPORTANTES

- Menor ênfase em documentação
- Menor ênfase em *big upfront design*
- Envolvimento constante do cliente
- Novas práticas de programação
 - ← Testes, refactoring, integração contínua, etc

MÉTODOS ÁGEIS QUE VAMOS ABORDAR AQUI

- Extreme Programming (XP)
- Scrum
- Kanban

ANTES DE COMEÇAR

- Nenhum processo é uma bala-de-prata
- Processo ajuda a não cometer certos "grandes erros"
- Processos não são adotados 100% igual ao manual
 - ← Bom senso é importante
 - ← Experimentação é importante

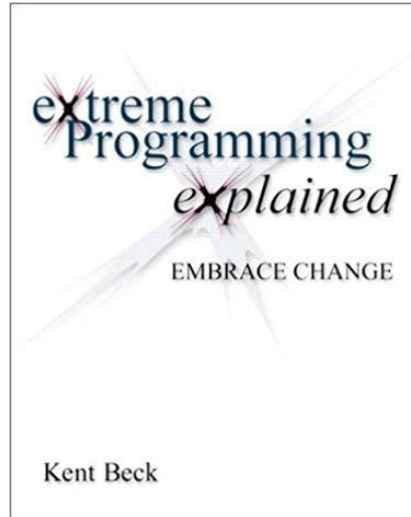


EXTREME PROGRAMMING (XP)

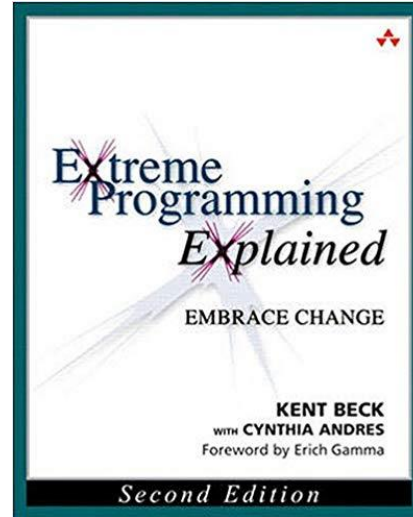
EXTREME PROGRAMMING



Kent Beck



1999



2004

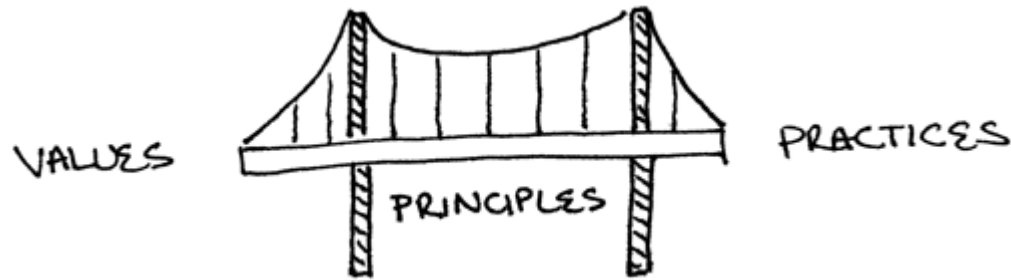


XP = VALORES + PRINCÍPIOS + PRÁTICAS

VALORES

- Comunicação
- Simplicidade
- Feedback
- Coragem
- Respeito
- Qualidade de Vida (semana 40 hrs)

XP = VALORES + PRINCÍPIOS + PRÁTICAS



PRINCÍPIOS

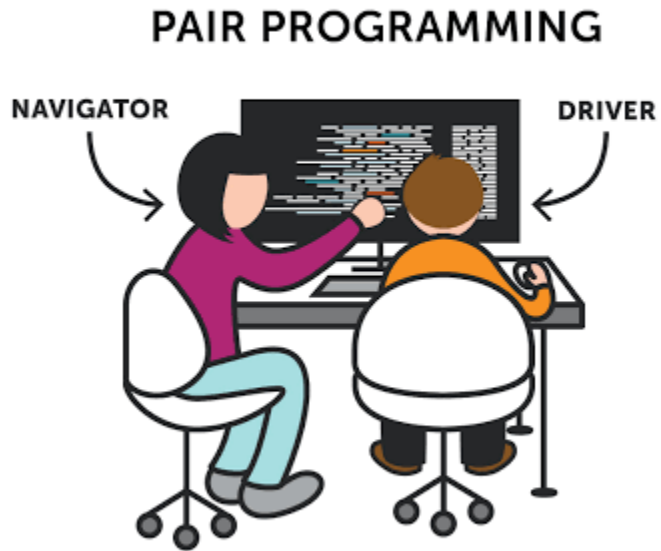
- Economicidade
- Melhorias Contínuas
- Falhas Acontecem
- Baby Steps
- Responsabilidade Pessoal



XP = VALORES + PRINCÍPIOS + PRÁTICAS

Práticas sobre o Processo de Desenvolvimento	Práticas de Programação	Práticas de Gerenciamento de Projetos
Representante dos Clientes Histórias de Usuário Iterações Releases Planejamento de Releases Planejamento de Iterações Planning Poker Slack	Design Incremental Programação Pareada Testes Automatizados Desenvolvimento Dirigido por Testes (TDD) Build Automatizado Integração Contínua	Ambiente de Trabalho Contratos com Escopo Aberto Métricas

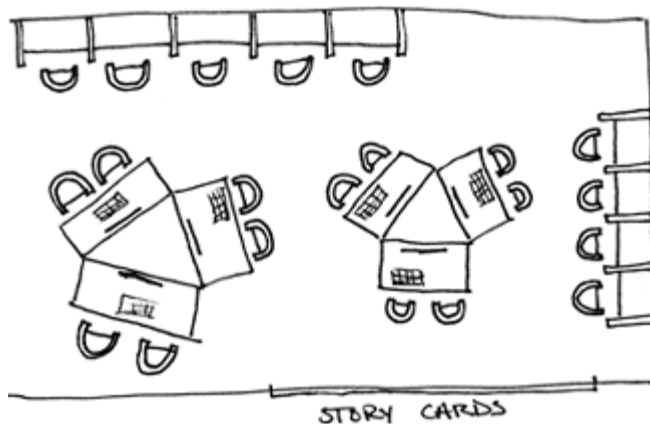
PAIR PROGRAMMING



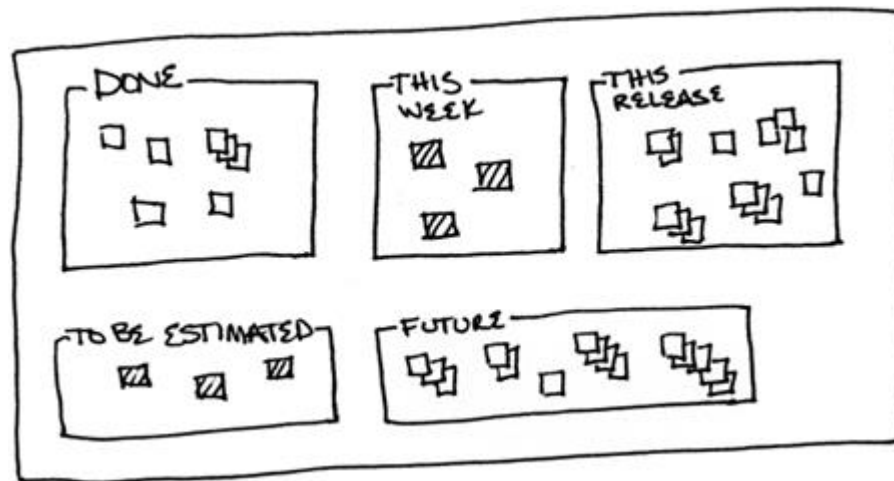
ESTUDO COM ENGENHEIROS DA MICROSOFT (2008)

- Vantagens:
 - ← Redução de bugs
 - ← Código de melhor qualidade
 - ← Disseminação de conhecimento
 - ← Aprendizado com os pares
- Desvantagem:
 - ← Custo

AMBIENTE DE TRABALHO



Ambiente de trabalho



Cartazes para "visualizar trabalho" em andamento

CONTRATOS DE SOFTWARE

- Basicamente, podem ser de dois tipos:
 - ← Escopo Fechado
 - ← Escopo Aberto (defendidos por XP)

CONTRATOS COM ESCOPO FECHADO

- Cliente define requisitos ("fecha escopo")
- Empresa desenvolvedora: preço + prazo

CONTRATOS COM ESCOPO ABERTO

- Escopo definido a cada iteração
- Pagamento por homem/hora
- Contrato renovado a cada iteração

CONTRATOS COM ESCOPO ABERTO

- Exige maturidade e acompanhamento do cliente
- Vantagens:
 - ← Privilegia qualidade
 - ← Não vai ser enganado ("entregar por entregar")
 - ← Pode mudar de fornecedor



SCRUM

SCRUM

- Proposto por Jeffrey Sutherland e Ken Schwaber (OOPSLA 1995)

SCRUM Development Process

Ken Schwaber

Advanced Development Methods

131 Middlesex Turnpike Burlington, MA 01803

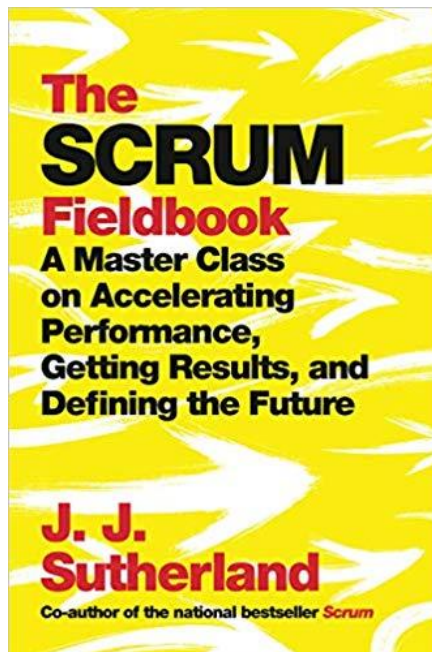
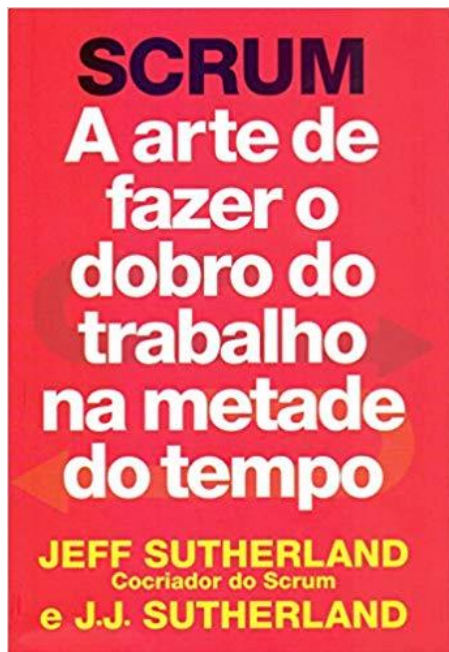
email virman@aol.com Fax: (617) 272-0555

ABSTRACT. *The stated, accepted philosophy for systems development is that the development process is a well understood approach that can be planned, estimated, and successfully completed. This has proven incorrect in practice. SCRUM assumes that the systems development process is an unpredictable, complicated process that can only be roughly described as an overall progression. SCRUM defines the systems development process as a loose set of activities that combines known, workable tools and techniques with the best that a development team can devise to build systems. Since these activities are loose, controls to manage the process and inherent risk are used. SCRUM is an enhancement of the commonly used iterative/incremental object-oriented development cycle.*

KEY WORDS: *SCRUM SEI Capability-Maturity-Model Process Empirical*

SCRUM

- Scrum é uma indústria: livros, consultoria, certificações, marketing



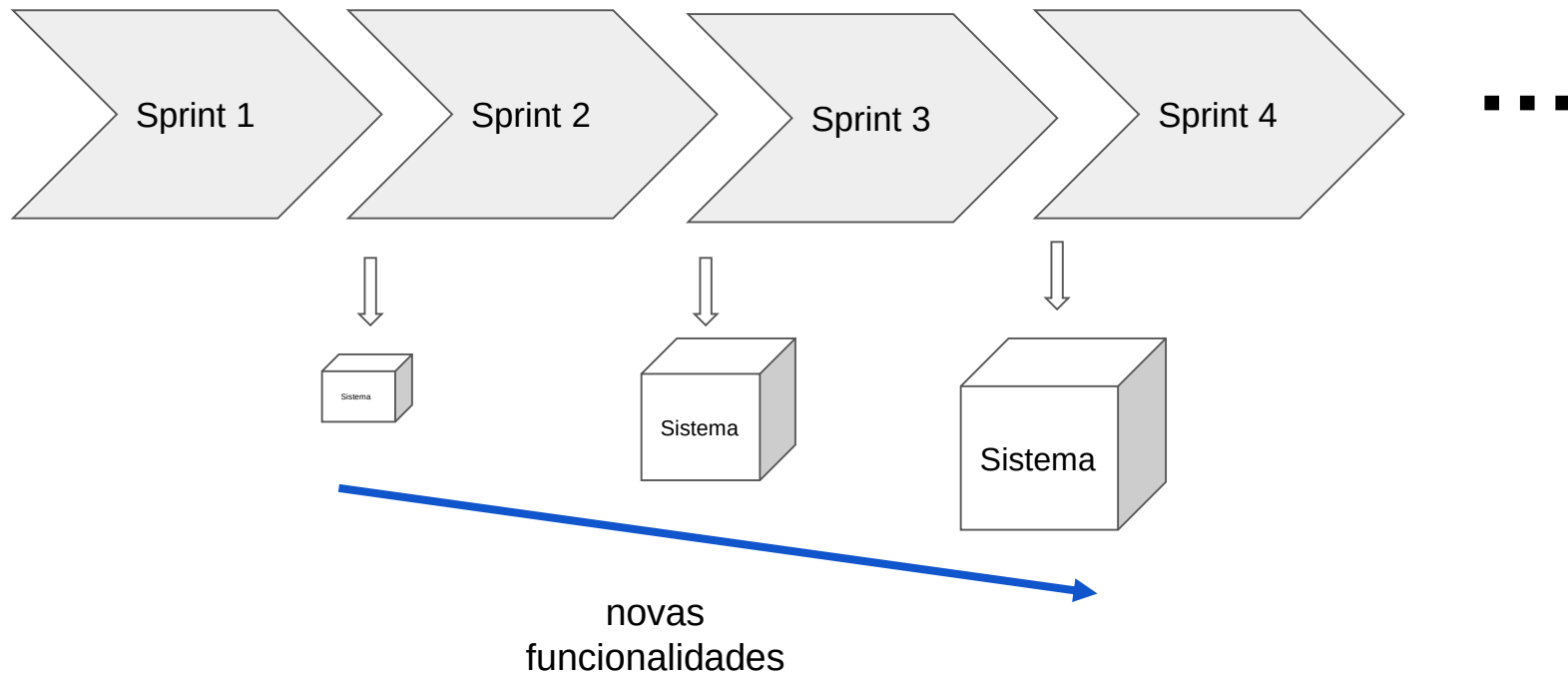
SCRUM VS XP

- Scrum não é apenas para projetos de software
 - ← Logo, não define práticas de programação, como XP
- Scrum define um "processo" mais rígido que XP
 - ← Eventos, papéis e artefatos bem claros

PRINCIPAL EVENTO: SPRINTS

- Como em qualquer método ágil, desenvolvimento é dividido em sprints (iterações)
- Duração de um sprint: até 1 mês, normalmente 15 dias

SPRINTS



O QUE SE FAZ EM UM SPRINT?

- Implementa-se algumas **histórias dos usuários**
- Histórias = funcionalidades (ou features) do sistema
- Exemplo: fórum de perguntas e respostas

Postar Pergunta

Um usuário, quando logado no sistema, deve ser capaz de postar perguntas. Como é um site sobre programação, as perguntas podem incluir blocos de código, os quais devem ser apresentados com um layout diferenciado.

Bem simples, deve caber em um post-it

QUEM ESCREVE AS HISTÓRIAS?

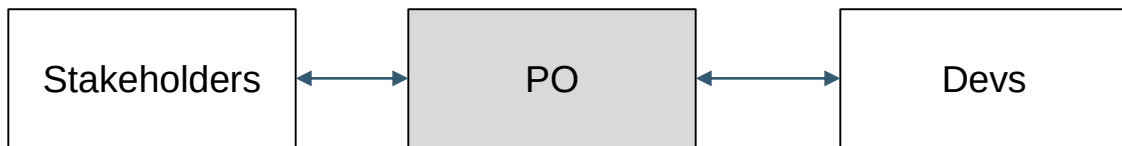
- **Product Owner (PO)**
- Membro obrigatório em times Scrum
- Especialista no domínio do sistema

ANTES (WATERFALL)



Observação: stakeholders são os clientes do sistema e qualquer outra parte interessada ou afetada por ele.
Exemplo: departamento jurídico é interessado no módulo de contratos de um sistema

HOJE (SCRUM)



- Durante sprint, PO explica histórias (requisitos) para devs
- Troca-se documentação formal e escrita por documentação verbal e informal
- Conversas entre PO e Devs

FUNÇÕES DE UM PO

- Escrever histórias dos usuários
- Explicar histórias para os devs, durante o sprint
- Definir "testes de aceitação" de histórias
- Priorizar histórias
- Manter o backlog do produto

BACKLOG DO PRODUTO

- Lista de histórias do usuário, que foram escritas pelo PO
- Duas características:
 - ↩ Priorizada: histórias do topo têm maior prioridade
 - ↩ Dinâmica: histórias podem sair e entrar, à medida que o sistema evolui

RESUMINDO

- Sprint (evento)
- PO e Devs (papéis)
- Backlog do produto (artefato)

QUAIS HISTÓRIAS VÃO ENTRAR NO PRÓXIMO SPRINT?

- Essa decisão é tomada no início do sprint
- Em uma reunião chamada de **planejamento do sprint**
- PO propõe histórias que gostaria de ver implementadas
- Devs decidem se têm **velocidade** para implementá-las

PLANEJAMENTO DO SPRINT

- 1a parte da reunião:
 - ↩ Definem-se as histórias do sprint
- 2a parte da reunião:
 - ↩ Histórias são quebradas em tarefas
 - ↩ Tarefas são alocadas a devs

EXEMPLO:

- História: Postar Perguntas
- Tarefas:
 - ← Projetar e testar a interface Web, incluindo layout, CSS templates, etc.
 - ← Instalar banco de dados, projetar e criar tabelas.
 - ← Implementar a camada de acesso a dados.
 - ← Implementar camada de controle, com operações para cadastrar, remover e atualizar perguntas.
 - ← Implementar interface Web.

BACKLOG DO SPRINT

- Lista de tarefas do sprint, com responsáveis e duração estimada



SPRINT ESTÁ PRONTO PARA COMEÇAR!



SCRUM: CONCEITOS COMPLEMENTARES

TIMES SCRUM

- Pequenos, do tamanho de "duas pizzas"
- 5 a 11 membros
 - ← 1 PO
 - ← 3 a 9 desenvolvedores
 - ← 1 Scrum Master

SCRUM MASTER

- Especialista em Scrum do time
 - ← Ajudar o time a seguir Scrum e seus eventos
- Removedor de "impedimentos" não-técnicos
 - ← Exemplo: desenvolvedores não têm máquinas boas
- Não é o chefe do time
- Pode pertencer a mais de um time

TIPOS DE PRODUCT OWNER

- Sistema acadêmico
- Cliente/contratante: Pró-reitoria de Graduação (prograd)
 - ↩ #1: desenv. interno $\Rightarrow$ PO: funcionário da prograd
 - ↩ #2: desenv. terceirizado $\Rightarrow$ PO: funcionário da prograd
 - ↩ #3: compra produto $\Rightarrow$ PO: "proxy" do cliente
- PO: próximo do problema



MAIS ALGUNS EVENTOS

REUNIÕES DIÁRIAS

- 15 minutos de duração; em pé. Cada participante diz:
 - ← o que ele fez ontem
 - ← o que pretende fazer hoje
 - ← e se está tendo alguma dificuldade
- Objetivos:
 - ← Melhorar comunicação
 - ← Antecipar problemas



SPRINT TERMINA COM DOIS EVENTOS: REVIEW E RETROSPECTIVA

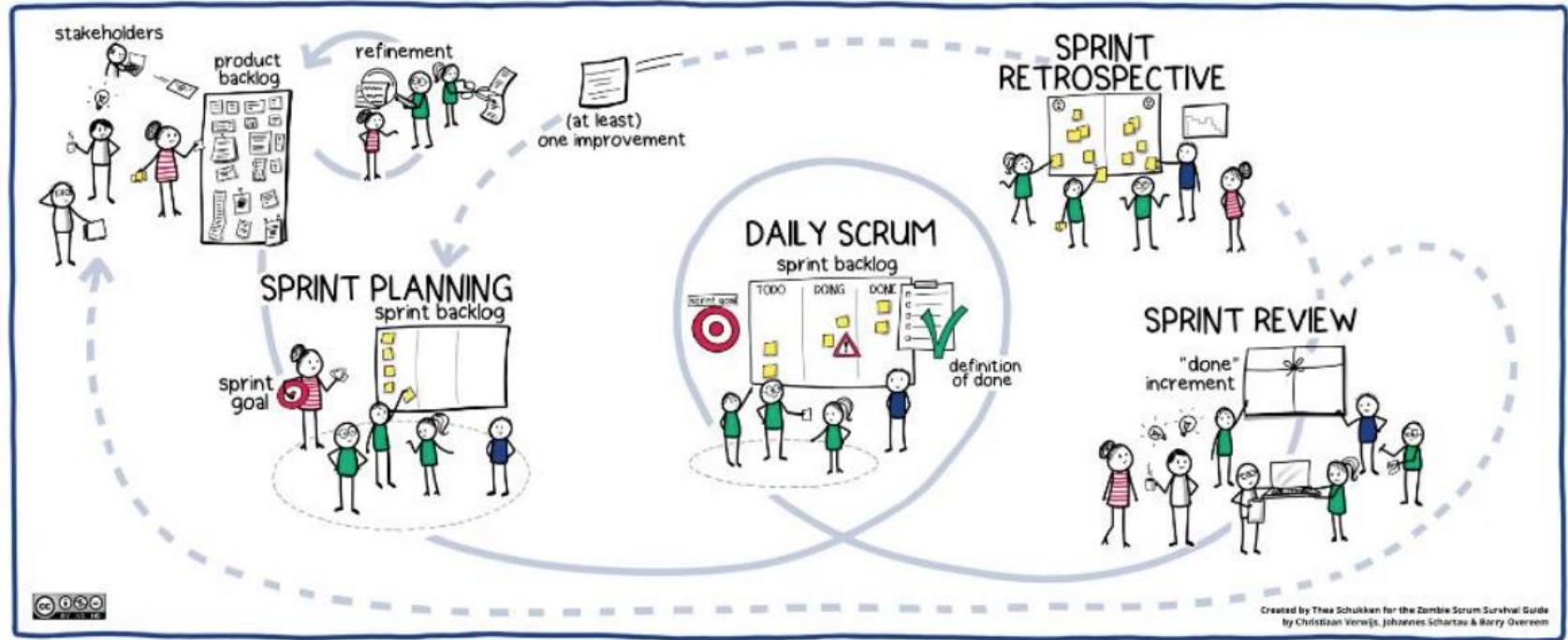
REVISÃO DO SPRINT

- Time mostra o resultado do sprint para PO e stakeholders
- Implementação das histórias pode ser:
 - ← Aprovada
 - ← Aprovada parcialmente
 - ← Reprovada
- Nos dois últimos casos, história volta para o backlog do produto

RETROSPECTIVA

- Último evento do sprint
- Time se reúne para decidir o que melhorar
 - ← O que deu certo?
 - ← Onde precisamos melhorar?
- Modelo mental: melhorias constantes
- Não é para "lavar a roupa suja"

RESUMO EM 1 SLIDE





MAIS ALGUNS CONCEITOS DE SCRUM

TIME-BOX

- Atividades têm uma duração bem definida

Evento	Time-box
Planejamento do Sprint	máximo de 8 horas
Sprint	menos de 1 mês
Reunião Diária	15 minutos
Revisão do Sprint	máximo de 4 horas
Retrospectiva	máximo de 3 horas

CRITÉRIOS PARA CONCLUSÃO DE HISTÓRIAS (DONE CRITERIA)

- Considerar qualidade externa e interna
- Qualidade externa:
 - ↩ Testes de aceitação
 - ↩ Testes não-funcionais (ex.: desempenho, usabilidade)
- Qualidade interna:
 - ↩ Testes de unidade
 - ↩ Revisão de código



MAIS ALGUNS ARTEFATOS DE SCRUM

SCRUM BOARD

Backlog	To Do	Doing	Testing	Done
     	        	       	   	 

SCRUM BOARD



EXEMPLO: PROJETO DA MOZILLA (USANDO GITHUB PROJECTS)

Smart Scheduling

Updated 13 days ago

Filter cards

100 Backlog

Bug 1632870 - Store test configuration in the task definition

Added by ahal

Bug 1667401 - Always schedule "new manifests" with manifest based bugbug optimizers

Added by ahal

Investigate Treeherder UI/UX changes for test filtering

Added by armenzg

Handle pushes containing multiple backouts

mozci#204 opened by marco-c

enhancement

When a task/group has inconsistent classifications, use the status of the task/group on the backout to figure it out

mozci#200 opened by marco-c

enhancement

Bug 1635921 - Use |mach try fuzzy| as a means to select configurations with

1 Next up

Reinstate integration tests

mozci#390 opened by marco-c

3 In progress

Build a dashboard to see schedulers results over time (both # of tests and durations)

Added by marco-c

Add a way to get durations of shadow scheduler tasks

mozci#102 opened by ahal

metrics

1 linked pull request

Bug 1639164 - "mach try auto" should select the best platforms to run manifests on

assigned: marco

Added by marco-c

222 Done

Make adr dependency optional

mozci#388 opened by marco-c

Bug 1671422 - Add durations to group_result actions in errorssummary formatter

assigned: ahal

Added by ahal

Cache results from data sources

mozci#368 opened by marco-c

enhancement

1 linked pull request

OOMs while analyzing some pushes

mozci#366 opened by marco-c

bug

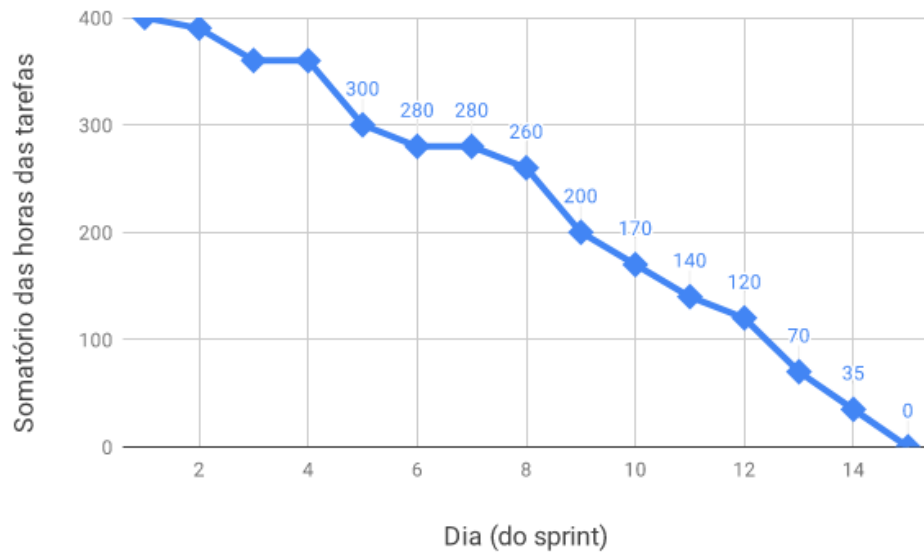
1 linked pull request

Add support for getting groups and groups results in the Treeherder data source

mozci#312 opened by marco-c

1 linked pull request

BURNDOWN CHART





ESTIMATIVA DE TAMANHO DE HISTÓRIAS DE USUÁRIOS

STORY POINTS

- Unidade (inteiro) para comparação do tamanho de histórias.

Exemplo de escala: 1, 2, 3, 5, 8, 13

História	Story Points
Cadastrar usuário	8
Postar perguntas	5
Postar respostas	3
Tela de abertura	5
Gamificar perguntas e respostas	5
Pesquisar perguntas e respostas	8
Adicionar tags em perguntas e respostas	5
Comentar perguntas e respostas	3

VELOCIDADE DE UM TIME

- Número de story points que o time consegue implementar em um sprint
- Definição de story points é "empírica"



KANBAN

KANBAN

- Origem na década de 50 no Japão
- Sistema de Produção da Toyota
- Manufatura Lean
- Produção just-in time

KANBAN = "CARTÃO VISUAL"



KANBAN EM DESENVOLVIMENTO DE SOFTWARE



KANBAN VS SCRUM

- Kanban é mais simples
- Não existem sprints
- Não é obrigatório usar papéis e eventos, incluindo:
 - ← Scrum master
 - ← Daily Scrum, Retrospectivas, Revisões
- Time define os papéis e eventos

KANBAN

"Grandes" colunas do quadro:
Passos

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas

1a sub-coluna:
em andamento

2a sub-coluna:
concluídas

Fluxo de trabalho (tempo)

KANBAN

- Ideia central: sistema pull
- Membros "puxam" trabalho:
 - a. Escolhem uma tarefa para trabalhar
 - b. Concluem tarefa (movem ela para frente no quadro)
 - c. Voltam para o passo (a)

tempo

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas



Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas
						

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	
						

tempo

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec. 	especificadas	em implementação	implementadas	em revisão	

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas    	em implementação	implementadas	em revisão	revisadas

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas □ □ □ □	em implementação	implementadas	em revisão	revisadas

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas □ □ □	em implementação □	implementadas	em revisão	revisadas

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas □ □ □	em implementação	implementadas □	em revisão	

tempo



EXEMPLO 2

EXEMPLO 2: ONTEM NO FINAL DO DIA

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
H3	em espec. H2	especificadas T6 T7 T8 T9	em implementação T4 T5	implementadas T3	em revisão T2	revisadas T1

HOJE NO FINAL DO DIA:

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
H3	em espec.	especificadas T8 T9 T10 T11 T12	em implementação T4 T5 T6 T7	implementadas	em revisão T3	revisadas T1 T2



KANBAN: LIMITES WIP

LIMITES WIP (WORK IN PROGRESS)

Backlog	Especificação 2		Implementação 5		Revisão de Código 3	
H3	em espec.	especificadas T8 T9 T10 T11 T12	em implementação T4 T5 T6 T7	implementadas	em revisão T3	revisadas T1 T2

LIMITES WIP (WORK IN PROGRESS)

- Número máximo de tarefas em um passo
- Contando: tarefas em andamento e concluídas

Backlog	Especificação 2		Implementação 5		Revisão de Código 3	
H3	em espec.	especificadas T8 T9 T10 T11 T12	em implementação T4 T5 T6 T7	implementadas	em revisão T3	revisadas T1 T2

Diagram illustrating WIP limits:

- Arrows point from the 'em implementação' and 'implementadas' columns to a blue oval containing circled numbers 4 and 0.

OBJETIVOS DOS LIMITES WIP

- Criar um fluxo de trabalho sustentável
 - ↩ Evitar que o time fique sobrecarregado de trabalho
 - ↩ WIP = "acordo" entre o time e a organização
 - ↩ Capacidade de trabalho de um time
- Evitar que o trabalho fique concentrado em um passo



FRASE COMUM EM KANBAN:

“PARE DE COMEÇAR E COMECE A TERMINAR”

EXEMPLO DE VIOLAÇÃO DE WIP

Backlog	Especificação (2)		Implementação (5)		Validação (3)	
			X	X		
			X	X		
			X	X		

- Esse quadro não é válido
- Existem 6 tarefas em Implementação
- Mas o WIP desse passo é 5

MAIS UM EXEMPLO: IMPLEMENTAÇÃO NO LIMITE

Backlog	Especificação (2)		Implementação (5)		Revisão (3)	
			X X	X X X		

- Talvez seja o momento de revisar algumas tarefas

WIP DO PASSO DE ESPECIFICAÇÃO

- Número de histórias em especificação + número de grupos de tarefas especificadas (linhas do quadro)
- Tarefas na mesma linha = resultantes da mesma história

Backlog	Especificação 2		Implementação 5		Revisão de Código 3	
	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas
H3		T8 T9 T10 T11 T12	T4 T5 T6 T7		T3	T1 T2

0

+1

+1

WIP DE REVISÃO DE CÓDIGO

- Conta apenas tarefas na primeira coluna (em revisão)

Backlog	Especificação 2		Implementação 5		Revisão de Código 3	
H3	em espec.	especificadas T8 T9 T10 T11 T12	em implementação T4 T5 T6 T7	implementadas	em revisão T3	revisadas T1 T2

1

Não contam para
fins de WIP

COMENTÁRIOS FINAIS SOBRE KANBAN

- Kanban é mais simples do que Scrum
- Kanban é mais adequado para times mais maduros
- Talvez: começar com Scrum e depois migrar para Kanban



QUANDO NÃO USAR MÉTODOS ÁGEIS?

QUANDO **NÃO** VALE A PENA USAR MÉTODOS ÁGEIS

- Condições do mercado são estáveis e previsíveis
- Requisitos são claros no início do projeto e irão permanecer estáveis
- Clientes não estão disponíveis para colaboração frequente
- Sistema semelhante já foi feito antes; logo, já se conhece a solução
- Problemas podem ser resolvidos sequencialmente
- Clientes não conseguem testar partes do produto, antes de completo
- Mudanças no final do projeto são caras ou impossíveis
- Impacto de mudanças provisórias pode ser catastrófico

REFERÊNCIAS

- PRESSMAN, R. S.. Engenharia de Software. Makron Books. 1995.
- Valente, M. T. Engenharia de Software Moderna. Editora Independente, 2020 (<https://engsoftmoderna.info/>)